מדריך: מיון תמונות למסמכים ותמונות

Oct 28, 2026 · @Adivi

מה צריך לפני שמתחילים

הסקריפט מעתיק כל תמונה לאחת משלוש תיקיות: documents (מסמכים), photos (תמונות) ו-uncertain (לא בטוח). כל העבודה נעשית על המחשב שלך, בלי התמונות המקוריות לא משתנות. להעלות תמונות לשום מקום.

- [ ] מחשב ווינדוס עם חיבור לאינטרנט (נדרש רק להתקנה ולהורדת המודל בפעם הראשונה).
- [ ] הקובץ sort_photos.py. אפשר להעביר אותו במייל, בדיסק און קי או ב-Google Drive, או ליצור אותו מהקוד שבנסח בסוף המסמך (שלב 2 מסביר איך).
- [ ] מקום פנוי בדיסק: בערך כמו הנפח של תיקיית התמונות, ועוד בערך 3GB לתוכנות ולמודל. כדי לבדוק: קליק ימני על תיקיית התמונות ← מאפיינים.
- [ ] כדי להעתיק אותו: פתח את התיקייה בסייר הקבצים, למשל D:\Pictures. הנתיב לתיקיית התמונות ולחץ על שורת הכתובת למעלה.
- [ ] מטען מחובר. ההרצה המלאה עלולה לקחת בערך חצי שעה עד שעה וחצי במחשב נייד ישן.

הוא תיקיית הפלט. החלף אותם בנתיבים D:\Sorted-הוא תיקיית התמונות ו D:\Pictures בכל המסמך שלך בכל פקודה.

שלב 1: התקנת Python (פעם אחת, בערך 5 דקות)

הוא התוכנה שמריצה את הסקריפט. אם הוא כבר מותקן במחשב בגרסה 3.9 ומעלה, דלג לשלב Python 2.

1. בדפדפן, היכנס ל-python.org/downloads.
2. הורד את Python 3.12, בקובץ שנקרא Windows installer (64-bit).
3. פתח את הקובץ שהורד.
4. בתחתית החלון הראשון סמן את התיבה **חשוב:** Add python.exe to PATH. בלי זה הפקודות בשלבים הבאים לא יעבדו.
5. לחץ Install Now וחכה לסיום. אם מופיעה הודעה Disable path length limit, לחץ עליה. אחר כך לחץ Close.

שלב 2: הכנת תיקייה והעברת הסקריפט

הסקריפט צריך תיקייה משלו, מחוץ לתיקיית התמונות.

1. פתח סייר קבצים (מקש Windows + E) והיכנס לכונן C.

2. צור תיקייה חדשה בשם PhotoSort, כך שהנתיב שלה יהיה C:\PhotoSort.
3. שים בתוכה את sort_photos.py.

העתק לתוכו את כל הקוד (Notepad), **אם אין לך את הקובץ במחשב הזה:** פתח את פנקס הרשימות מהסעיף "נספח: קוד הסקריפט" בסוף המסמך, ושמור באופן הבא:

1. קובץ ← שמור בשם.
2. בחר את התיקייה C:\PhotoSort.
3. ב"שמור כסוג" בחר "כל הקבצים (*.*)".
4. בשם הקובץ כתוב sort_photos.py.
5. בקידוד בחר UTF-8 ולחץ שמור.

ככה הקובץ לא יישמר בטעות בשם sort_photos.py.txt.

שלב 3: פתיחת שורת פקודה בתיקייה

כל הפקודות מכאן והלאה נכתבות בחלון שחור שנפתח בשלב הזה. אל תסגור אותו עד הסוף.

1. בסייר הקבצים, היכנס לתיקייה C:\PhotoSort.
2. לחץ על שורת הכתובת למעלה, מחק את מה שכתוב שם, כתוב cmd ולחץ Enter. ייפתח חלון שחור שכבר נמצא בתיקייה הנכונה.
3. כתוב את הפקודה הזו ולחץ Enter:

```
python --version
```

אם מופיע משהו כמו Python 3.12.x, הכול תקין ואפשר להמשיך.

במקום py אם זה עובד, כתוב py --version. אם נפתחת חנות מיקרוסופט או מופיעה שגיאה, נסה בכל הפקודות במסמך, ו-py -m pip במקום pip.

שלב 4: התקנת הספריות (פעם אחת, 5 עד 15 דקות)

הספריות הן הכלים שהסקריפט צריך: מודל הבינה המלאכותית וקריאת תמונות. באותו חלון שחור, הרץ את שתי הפקודות האלה, אחת אחרי השנייה. המתן שהראשונה תסתיים לפני שמריצים את השנייה.

פקודה ראשונה, מנוע הבינה המלאכותית. זו גרסה למעבד בלבד, קטנה בהרבה מהרגילה:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

פקודה שנייה, המודל וקריאת תמונות:

```
pip install open_clip_torch pillow pillow-heif tqdm
```

אזהרות בצהוב על .Successfully installed-ההתקנה הצליחה אם בסוף מופיעה שורה שמתחילה ב
.אפשר להתעלם מהן pip גרסת

שלב 5: הרצת ניסיון על 300 תמונות (בערך 5 דקות)

הניסיון מסווג 300 תמונות בלבד ולא מעתיק את הקבצים. המטרה שלו היא לוודא שהכול עובד ולראות
.כמה מדויקים הסיווגים

:באותו חלון שחור הרץ את הפקודה הבאה. החלף את הנתיבים בנתיבים שלך והשאר את המירכאות

```
python sort_photos.py "D:\Pictures" "D:\Sorted" --dry-run --limit 300
```

- .הנתיב הראשון הוא תיקיית התמונות. הסקריפט עובר גם על כל תתי-התיקיות שלה
- .הנתיב השני הוא תיקיית הפלט. היא תיווצר לבד, ואסור שתהיה בתוך תיקיית התמונות
- .זה קורה רק פעם אחת .(600MB בערך) בהרצה הראשונה המודל יורד מהאינטרנט
- כמה תמונות בשנייה. את ,img/s בזמן הריצה מופיע פס התקדמות. רשום לעצמך את המספר שלפני זמן ההרצה המלאה מחשבים כך: 40,000 מחולק במספר הזה נותן את מספר השניות. למשל, 15 .תמונות בשנייה זה בערך 45 דקות

.עם כמה תמונות נכנסו לכל קטגוריה ,Done this run-בסוף מופיעה שורה שמתחילה ב

שלב 6: בדיקת תוצאות הניסיון

.את ההרצה המלאה מריצים רק אחרי שהסיווגים בניסיון נראים סבירים

1. .באקסל D:\Sorted\results.csv פתח את הקובץ
2. confidence מופיע הסיווג, ובעמודה category מופיע נתיב התמונה. בעמודה source בעמודה .הביטחון, מ-0 עד 1
3. .בחר 10 עד 20 שורות, פתח את התמונות עצמן ובדוק שהסיווג נכון

מה עושים	מה רואים
ממשיכים כרגיל	רוב הסיווגים נכונים
לפקודה בשלב 7 0.6 threshold-- מוסיפים	הרבה תמונות סומנו uncertain
ומשווים no-quantize-- מריצים שוב את הניסיון עם	הרבה טעויות במסמכים ובתמונות
ראו "שיפור הסיווג" בסעיף פתרון הבעיות	סוג מסוים מסווג לא נכון שוב ושוב

הסקריפט זוכר בקובץ הזה אילו **.results.csv לפני שממשיכים סגור את אקסל ומחק את הקובץ**
.תמונות כבר עיבד. אם לא מוחקים אותו, הוא ידלג על 300 תמונות הניסיון ולא יעתיק אותן

שלב 7: הרצה מלאה (בערך חצי שעה עד שעה וחצי)

כדאי להריץ כשלא צריך את המחשב לדברים אחרים, כי הסקריפט משתמש בכל כוח העיבוד.

לפני שמתחילים:

- [ ] חבר את המחשב למטען.
- [ ] בטל מצב שינה: הגדרות ← מערכת ← חשמל. תחת "כשמחובר לחשמל", קבע שהמחשב לא ייכנס למצב שינה לעולם.
- [ ] ודא שמחקת את results.csv מהניסיון.

הפקודה, באותו חלון שחור (או בחלון חדש שפתחת כמו בשלב 3):

```
python sort_photos.py "D:\Pictures" "D:\Sorted"
```

אם החלטת בשלב 6 להוריד את הסף, הפקודה היא:

```
python sort_photos.py "D:\Pictures" "D:\Sorted" --threshold 0.6
```

- הקבצים **מועתקים**, ותיקיית המקור נשארת כמו שהיא.
- כדי להמשיך, הרץ שוב את אותה פקודה בדיוק, Ctrl+C. **עצירה והמשכה:** אפשר לעצור בכל רגע עם והסקריפט ימשיך מאיפה שעצר. זה עובד גם אם המחשב נכבה.
- ההרצה נגמרה כשמופיעה השורה Done this run.

שלב 8: מעבר על התוצאה

יהיו שלוש תיקיות. בכל אחת מהן מבנה תתי-התיקיות המקורי נשמר D:\Sorted בסיום, בתוך.

תיקייה	מה יש בה	מה עושים
documents	מסמכים, קבלות, צילומי מסך, פתקים	מעבר מהיר לאיתור תמונות שנכנסו בטעות
photos	אנשים, משפחה, נופים, אוכל וכו'	מעבר מהיר לאיתור מסמכים שנכנסו בטעות
uncertain	תמונות שהמודל לא היה בטוח לגביהן	מעבר ידני והעברת כל תמונה לתיקייה הנכונה

קבצים פגומים או כאלה שלא ניתן לפתוח לא מועתקים. הם מסומנים error בקובץ results.csv.

מחיקת המקור: רק אחרי שעברת על התוצאה ואתה מרוצה. ודא קודם שמספר הקבצים ב-D:\Sorted דומה למספר במקור (קליק ימני על התיקייה ← מאפיינים). עדיף לשמור גיבוי של המקור לתקופה מסוימת.

פתרון בעיות ואפשרויות נוספות

בעיה	פתרון
'python' is not recognized	בשלב 1. נסה py לא סומנה התיבה Add to PATH או התקן מחדש וסמן את התיבה ,python במקום
'pip' is not recognized	כתוב python -m pip במקום pip
can't open file sort_photos.py	החלון השחור לא פתוח ב-C:\PhotoSort, או שהקובץ חזור לשלבים 2 ו-3. נשמר בשם sort_photos.py.txt
Source folder not found	הנתיב לתיקיית התמונות שגוי. העתק אותו משורת הכתובת בסייר והשאר את המירכאות
שגיאה בהורדת המודל	בדוק חיבור לאינטרנט והרץ שוב
המחשב נתקע או מופיעה שגיאת זיכרון (memory)	הוסף לפקודה --batch-size 8
שגיאה שמזכירה quantize	הוסף לפקודה --no-quantize
כל שגיאה אחרת	צלם את המסך או העתק את השורות האדומות ושלח ל-Claude

שיפור הסיווג: בתחילת הקובץ sort_photos.py יש רשימה בשם CATEGORIES, עם תיאורים באנגלית. אם סוג מסוים מסווג לא נכון, הוסף לרשימה המתאימה שורה שמתארת אותו, למשל "a photo of a medical prescription" תחת documents. 5 שמור את הקובץ והרץ שוב את הניסיון בשלב.

אפשרויות שאפשר להוסיף לפקודה:

אפשרות	מה היא עושה
--dry-run	מסווג בלי להעתיק קבצים
--limit 300	מעבד רק את המספר הזה של תמונות
--threshold 0.6	סף הביטחון (ברירת מחדל 0.75). נמוך יותר אומר פחות תמונות ב-uncertain
--move	מזיז במקום להעתיק וחוסך מקום בדיסק. רק אחרי שאתה סומך על התוצאות
--batch-size 8	מעבד פחות תמונות בכל פעם. משתמשים בזה אם אין מספיק זיכרון
--no-quantize	מבטל את האצת int8. איטי יותר, לפעמים מדויק יותר

נספח: קוד הסקריפט

מעתיקים אותו רק אם אין לך את הקובץ במחשב האחר (שלב 2 .sort_photos.py זה הקוד המלא של מסביר איך לשמור אותו). צריך להעתיק את כל הקוד, כולל הרווחים בתחילת השורות.

```
"""
sort_photos.py - Sort a large photo collection into "documents" vs "photos"
using a zero-shot CLIP model. Runs fully offline after the first model
download,
CPU-only friendly (old laptops are fine, just slower).

Output folders (inside --dst):
    documents/   - photographed documents, receipts, screenshots, notes,
IDs...
    photos/      - real photos (people, family, places, food, etc.)
    uncertain/   - the model wasn't confident enough -> check these by hand

The original folder structure is preserved under each output folder, so files
with the same name never overwrite each other.

Usage examples (Windows):
    python sort_photos.py "D:\\Pictures" "D:\\Sorted" --dry-run --limit 300
    python sort_photos.py "D:\\Pictures" "D:\\Sorted"
    python sort_photos.py "D:\\Pictures" "D:\\Sorted" --move

It is safe to stop (Ctrl+C) and run again: already-processed files are
skipped
(tracked in results.csv inside --dst).
"""

import argparse
import csv
import os
import shutil
import sys
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

# ---------------------------------------------------------------------------
# Prompts: each category is a list of text descriptions. A picture's score
for
# a category is the sum of the probabilities of its prompts.
# Feel free to add/remove prompts if you see systematic mistakes.
# ---------------------------------------------------------------------------
CATEGORIES = {
    "documents": [
```

```
        "a photo of a document",
        "a photo of a printed page with text",
        "a photo of a receipt",
        "a photo of an invoice or a bill",
        "a photo of a form",
        "a photo of a handwritten note",
        "a photo of an ID card",
        "a photo of a whiteboard with writing",
        "a screenshot of a phone or computer screen",
        "a photo of a page from a book",
    ],
    "photos": [
        "a photo of people",
        "a family photo",
        "a selfie",
        "a photo of a child",
        "a group photo at an event",
        "a photo of a landscape",
        "a photo of a city street",
        "a photo of food",
        "a photo of an animal",
        "a photo of a room or a house",
    ],
}

IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp", ".tif", ".tiff",
".gif", ".heic", ".heif"}

MODEL_NAME = "ViT-B-32"               # light model, good for CPU
MODEL_WEIGHTS = "laion2b_s34b_b79k"


def parse_args():
    p = argparse.ArgumentParser(description="Sort photos into documents /
photos / uncertain using CLIP.")
    p.add_argument("src", help="Folder with the images (scanned
recursively)")
    p.add_argument("dst", help="Output folder (must NOT be inside src)")
    p.add_argument("--move", action="store_true", help="Move files instead of
copying (default: copy)")
    p.add_argument("--dry-run", action="store_true", help="Only classify and
write results.csv, don't copy/move")
    p.add_argument("--threshold", type=float, default=0.75,
                   help="Minimum confidence for a decision; below it ->
uncertain (default 0.75)")
    p.add_argument("--batch-size", type=int, default=32, help="Images per
```

```
batch (default 32)")
    p.add_argument("--no-quantize", action="store_true",
                   help="Disable int8 speed-up (use only if you get errors or
bad results)")
    p.add_argument("--workers", type=int, default=0,
                   help="Threads for loading images (default: number of CPU
cores)")
    p.add_argument("--threads", type=int, default=0, help="CPU threads for
torch (default: all cores)")
    p.add_argument("--limit", type=int, default=0, help="Process only the
first N new images (for testing)")
    return p.parse_args()


def find_images(src: Path, dst: Path):
    dst_resolved = dst.resolve()
    for root, dirs, files in os.walk(src):
        root_path = Path(root)
        # never descend into the output folder
        dirs[:] = [d for d in dirs if (root_path / d).resolve() !=
dst_resolved]
        for f in files:
            if Path(f).suffix.lower() in IMAGE_EXTS:
                yield root_path / f


def load_done(csv_path: Path):
    done = set()
    if csv_path.exists():
        with open(csv_path, newline="", encoding="utf-8") as fh:
            for row in csv.DictReader(fh):
                done.add(row["source"])
    return done


def unique_target(target: Path) -> Path:
    if not target.exists():
        return target
    stem, suffix, i = target.stem, target.suffix, 1
    while True:
        candidate = target.with_name(f"{stem}_{i}{suffix}")
        if not candidate.exists():
            return candidate
        i += 1
```

```
def main():
    args = parse_args()
    src = Path(args.src)
    dst = Path(args.dst)

    if not src.is_dir():
        sys.exit(f"Source folder not found: {src}")
    if dst.resolve() == src.resolve():
        sys.exit("dst must be different from src")
    dst.mkdir(parents=True, exist_ok=True)

    # Heavy imports after argument checks so --help is fast
    import torch
    import open_clip
    from PIL import Image, ImageOps
    from tqdm import tqdm

    try:
        from pillow_heif import register_heif_opener
        register_heif_opener()
    except ImportError:
        pass  # HEIC files will just fail to open and be marked as errors

    Image.MAX_IMAGE_PIXELS = None  # allow very large scans

    if args.threads > 0:
        torch.set_num_threads(args.threads)

    csv_path = dst / "results.csv"
    done = load_done(csv_path)

    print("Scanning for images...")
    all_images = [p for p in find_images(src, dst) if str(p) not in done]
    if args.limit > 0:
        all_images = all_images[: args.limit]
    print(f"{len(done)} already processed, {len(all_images)} to process.")
    if not all_images:
        return

    print(f"Loading model {MODEL_NAME} (first run downloads ~600MB)...")
    model, _, preprocess = open_clip.create_model_and_transforms(MODEL_NAME,
pretrained=MODEL_WEIGHTS)
    tokenizer = open_clip.get_tokenizer(MODEL_NAME)
    model.eval()

    # Pre-compute text embeddings once
```

```
    prompts, prompt_cat = [], []
    for cat, plist in CATEGORIES.items():
        for t in plist:
            prompts.append(t)
            prompt_cat.append(cat)
    cat_names = list(CATEGORIES.keys())
    cat_index = torch.tensor([cat_names.index(c) for c in prompt_cat])

    with torch.inference_mode():
        text_feat = model.encode_text(tokenizer(prompts))
        text_feat = text_feat / text_feat.norm(dim=-1, keepdim=True)

    # Speed-up 1: int8 quantization of the image model's linear layers.
    # ViT is mostly linear layers, so this is typically ~1.5-2x faster on
CPU.
    if not args.no_quantize:
        try:
            model.visual = torch.ao.quantization.quantize_dynamic(
                model.visual, {torch.nn.Linear}, dtype=torch.qint8)
            print("int8 speed-up enabled")
        except Exception as e:
            print(f"int8 speed-up not available on this machine ({e});
continuing without it")

    def load(path: Path):
        try:
            with Image.open(path) as im:
                # Speed-up 2: decode JPEGs directly at ~1/8 size (CLIP only
needs 224px).
                # Must be called before anything else touches the pixels.
                im.draft("RGB", (224, 224))
                im = ImageOps.exif_transpose(im)   # respect phone rotation
                im = im.convert("RGB")
                return preprocess(im)
        except Exception as e:  # corrupt / unsupported file
            return e

    new_file = not csv_path.exists()
    csv_fh = open(csv_path, "a", newline="", encoding="utf-8")
    writer = csv.writer(csv_fh)
    if new_file:
        writer.writerow(["source", "category", "confidence",
"documents_score", "photos_score", "target"])

    counts = {"documents": 0, "photos": 0, "uncertain": 0, "error": 0}
    workers = args.workers or (os.cpu_count() or 4)
```

```
    pool = ThreadPoolExecutor(max_workers=workers)
    batches = [all_images[i: i + args.batch_size] for i in range(0,
len(all_images), args.batch_size)]

    def submit(batch):
        return [pool.submit(load, p) for p in batch]

    try:
        with tqdm(total=len(all_images), unit="img") as bar:
            # Speed-up 3: load the NEXT batch from disk while the model works
on this one.
            next_futures = submit(batches[0])
            for b_idx, batch_paths in enumerate(batches):
                loaded = [f.result() for f in next_futures]
                if b_idx + 1 < len(batches):
                    next_futures = submit(batches[b_idx + 1])

                good_paths, tensors = [], []
                for pth, item in zip(batch_paths, loaded):
                    if isinstance(item, Exception):
                        writer.writerow([str(pth), "error", "", "", "",
str(item)[:200]])
                        counts["error"] += 1
                    else:
                        good_paths.append(pth)
                        tensors.append(item)

                if tensors:
                    with torch.inference_mode():
                        img_feat = model.encode_image(torch.stack(tensors))
                        img_feat = img_feat / img_feat.norm(dim=-1,
keepdim=True)
                        probs = (100.0 * img_feat @
text_feat.T).softmax(dim=-1)
                        cat_scores = torch.zeros(len(tensors),
len(cat_names))
                        cat_scores.index_add_(1, cat_index, probs)

                    for pth, scores in zip(good_paths, cat_scores):
                        conf, idx = scores.max(dim=0)
                        conf = float(conf)
                        category = cat_names[int(idx)] if conf >=
args.threshold else "uncertain"
                        counts[category] += 1

                        target_str = ""
```

```
                        if not args.dry_run:
                            rel = pth.relative_to(src)
                            target = unique_target(dst / category / rel)
                            target.parent.mkdir(parents=True, exist_ok=True)
                            if args.move:
                                shutil.move(str(pth), str(target))
                            else:
                                shutil.copy2(str(pth), str(target))
                            target_str = str(target)

                        writer.writerow([
                            str(pth), category, f"{conf:.3f}",
                            f"
{float(scores[cat_names.index('documents')]):.3f}",
                            f"
{float(scores[cat_names.index('photos')]):.3f}",
                            target_str,
                        ])

                csv_fh.flush()
                bar.update(len(batch_paths))
    except KeyboardInterrupt:
        print("\nStopped. Run the same command again to continue where you
left off.")
    finally:
        pool.shutdown(wait=False, cancel_futures=True)
        csv_fh.close()

    print("\nDone this run:", ", ".join(f"{k}: {v}" for k, v in
counts.items()))
    print(f"Full log: {csv_path}")


if __name__ == "__main__":
    main()
```